



END OF SEMESTER PRACTICAL GROUP PROJECT

End-to-End Secure Big Data Pipeline

By: Group 17

Betelhem Getachew Kebede – 670549

Zakariya Shafi Muhumed – 668596

Chiadika Somto Elue - 670280

Instructor: Richard Karanja

Discipline: DSA4030UA(Big Data Security)

SUMMER SEMESTER 2026

Contents

1. Executive Summary	4
2. System Architecture Diagram	4
3. Implementation Report	6
3.1 Environment Setup	6
3.2 Database Design	7
3.3 Dataset Generation	7
3.4 Authentication	8
3.5 Role-Based Access Control (RBAC)	8
3.6 Dataset Validation	9
3.7 SHA-256 Hash Generation	10
3.8 Secure Import Pipeline	11
3.9 Audit Logging	12
3.10 Dataset Integrity Verification	12
3.11 Secure Data Export	13
3.12 Security Monitoring (Grafana)	14
3.13 Desktop Graphical User Interface	15
4. Security Testing Matrix	16
Test 1: Authentication	16
Test 2: Role-Based Access Control (RBAC)	16
Test 3: Sensitive Data Encryption	18
Test 4: Dataset Integrity Verification	18
Test 5: Audit Logging	19
Test 6: Secure End-to-End Pipeline Execution	20
Test 7: Dataset Validation	20
Test 8: Bulk Data Export Detection	21
Test 9: Security Events Monitoring	22
5. Simulated Security Incidents	24
Incident 1: Unauthorized Login Attempt	24
Incident 2: Privilege Escalation Attempt	24
Incident 3: Dataset Tampering	25
6. Evidence Portfolio	27
6.1 Environment & Connectivity	27
6.2 Additional Authentication Evidence	27
6.3 Additional RBAC Evidence	27
6.4 Additional Encryption Evidence	27
6.5 Additional Audit Logging Evidence	28
6.6 Additional Pipeline Execution Evidence	29
6.7 Grafana Monitoring Dashboard: Full Panel Set	30
6.8 Evidence Organization	33
7. Risk Assessment Table	34
8. Conclusion and Recommendations	35

8.1 Vulnerabilities Identified	35
8.2 Remaining Risks	35
8.3 Recommended Improvements	35
8.4 Overall Assessment	35
9. Appendices	36
Appendix A: Source Code	36
Appendix B: Evidence Index	36
Appendix C: Demo Accounts Used in Testing	36

1. Executive Summary

Group 17 was tasked with securing the full lifecycle of a client's big data environment: collecting data, storing it securely, controlling access, monitoring activity, and protecting data integrity. A working, end-to-end secure data pipeline was built to ingest customer records, authenticate and authorize users by role, encrypt sensitive fields at rest, verify data integrity using cryptographic hashing, log every action for audit purposes, and monitor the system in real time.

The environment runs as three Dockerized services: PostgreSQL 17, pgAdmin 4, and Grafana OSS, orchestrated with docker-compose. The application layer is written in Python and implements bcrypt password hashing for authentication, a three-tier Role-Based Access Control model (Admin, Analyst, Auditor), Fernet symmetric encryption for personally identifiable customer fields, SHA-256 hashing for dataset integrity verification, and a centralized audit_logs / security_events schema that records every login, permission check, validation, import, export, and integrity check performed in the system. A synthetic dataset of 100,000 Kenyan customer records was generated using the Python Faker library to exercise the pipeline at scale.

Beyond the command-line interface originally planned, a working CustomTkinter desktop GUI was also delivered, reusing the same backend modules so that every security control (authentication, RBAC, encryption, integrity checks) is enforced identically regardless of which interface a user chooses.

Nine security tests were executed against the system, covering authentication, role-based access control, sensitive data encryption, dataset integrity verification, audit logging, full pipeline execution, dataset validation, bulk export detection, and security event monitoring. All nine tests behaved as designed, including correctly triggering failures and warnings under adverse conditions (wrong credentials, unauthorized actions, tampered datasets, and oversized exports). Three simulated security incidents (an unauthorized login attempt, a privilege escalation attempt, and dataset tampering) were also demonstrated and were correctly detected, logged, and blocked where appropriate.

This report documents the system architecture, implementation details, full testing evidence, an assessment of residual risks, and recommendations for hardening the system further before any production use.

2. System Architecture Diagram

The diagram below shows the major components of the system and how they connect: the client layer (CLI and desktop GUI), the Python application layer that implements every security control, the Docker network hosting PostgreSQL, pgAdmin, and Grafana, and the local filesystem used for the encryption key, stored hashes, source datasets, and exports.

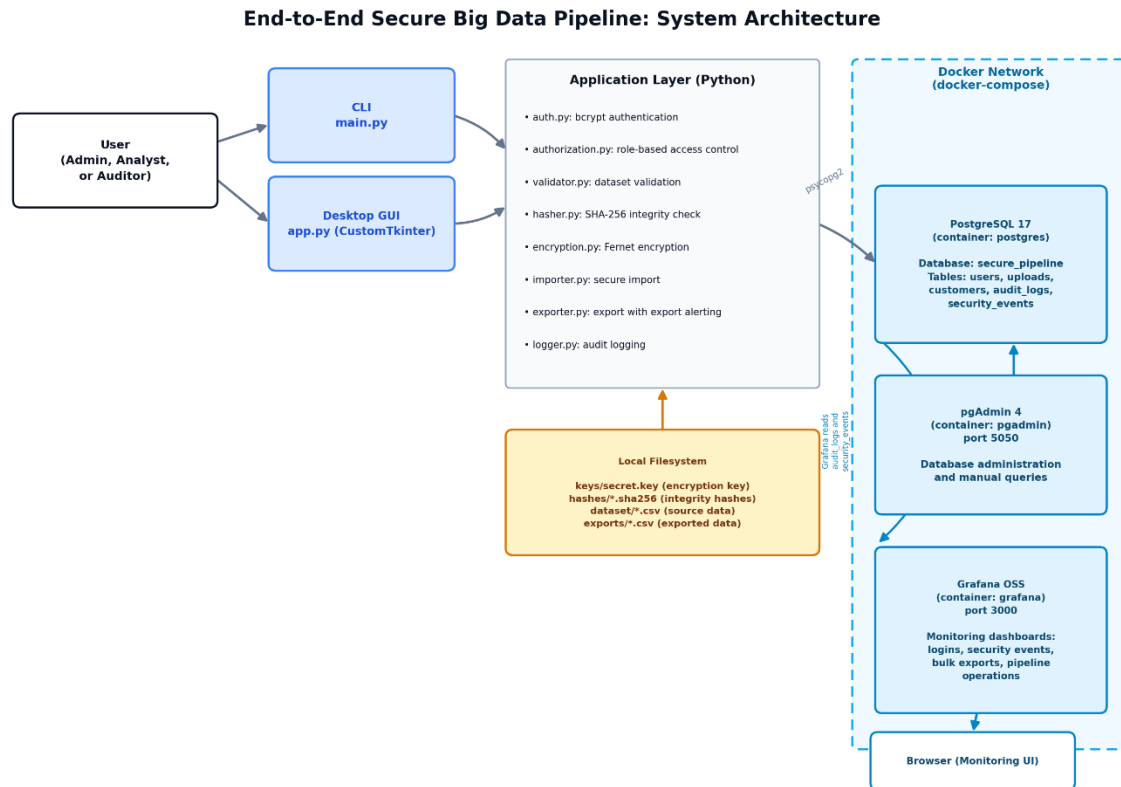


Figure 2.1: System architecture: client layer, application/security layer, and Dockerized data/monitoring layer.

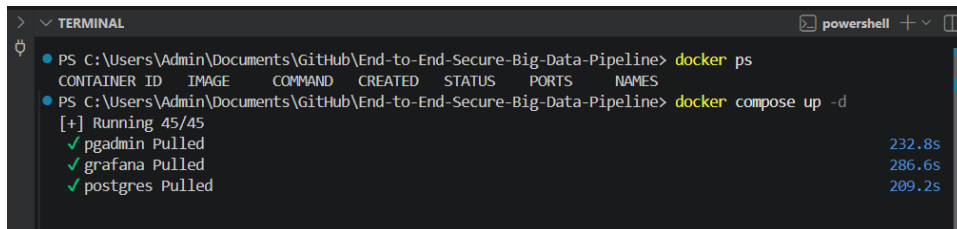
Component summary

- CLI (main.py) and Desktop GUI (app.py): two interchangeable front ends. Both call the exact same backend modules, so security behavior is identical either way.
- Application layer: auth.py (bcrypt), authorization.py (RBAC), validator.py, hasher.py (SHA-256), encryption.py (Fernet), importer.py, exporter.py, and logger.py.
- PostgreSQL 17 (Docker container) hosts the secure_pipeline database: users, uploads, customers, audit_logs, and security_events tables.
- pgAdmin 4 (Docker container, port 5050): used for direct database administration and manual verification queries during testing.
- Grafana OSS (Docker container, port 3000): reads audit_logs and security_events directly from PostgreSQL to render real-time dashboards (login activity, security events, bulk export alerts, pipeline operations).
- Local filesystem: keys/secret.key (Fernet key), hashes/*.sha256 (stored integrity hashes), dataset/*.csv (source data), exports/*.csv (export output).

3. Implementation Report

3.1 Environment Setup

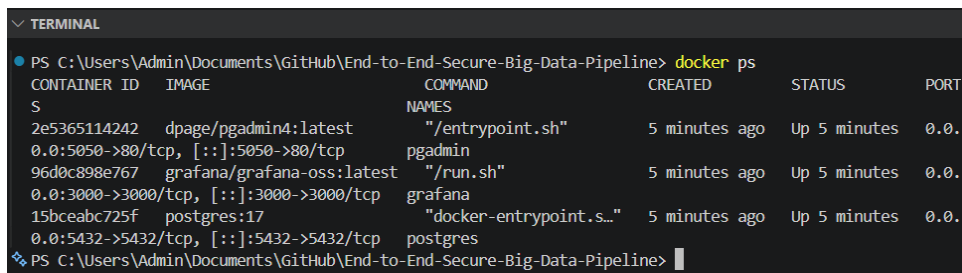
The environment was containerized using Docker Compose, running three services: PostgreSQL 17 (the primary data store), pgAdmin 4 (database administration UI), and Grafana OSS (security and operations monitoring). Database connection parameters are centralized in `config.py`, and `database.py` exposes a single reusable `get_connection()` function used by every other module.



```

PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> docker compose up -d
[+] Running 45/45
 ✓ pgadmin Pulled                                232.8s
 ✓ grafana Pulled                                286.6s
 ✓ postgres Pulled                               209.2s
  
```

Figure 3.1: `docker compose up -d` pulling and starting the `pgadmin`, `grafana`, and `postgres` images.



```

PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
2e5365114242   dpape/pgadmin4:latest               "/entrypoint.sh"        5 minutes ago Up 5 minutes   0.0.0.0:5050->80/tcp, [::]:5050->80/tcp   pgadmin
96d0c898e767   grafana/grafana-oss:latest          "/run.sh"                5 minutes ago Up 5 minutes   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   grafana
15bceabc725f   postgres:17                         "docker-entrypoint.s..." 5 minutes ago Up 5 minutes   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres
  
```

Figure 3.2: `docker ps` confirming all three containers are up and healthy.



Figure 3.3: `pgAdmin` login screen.

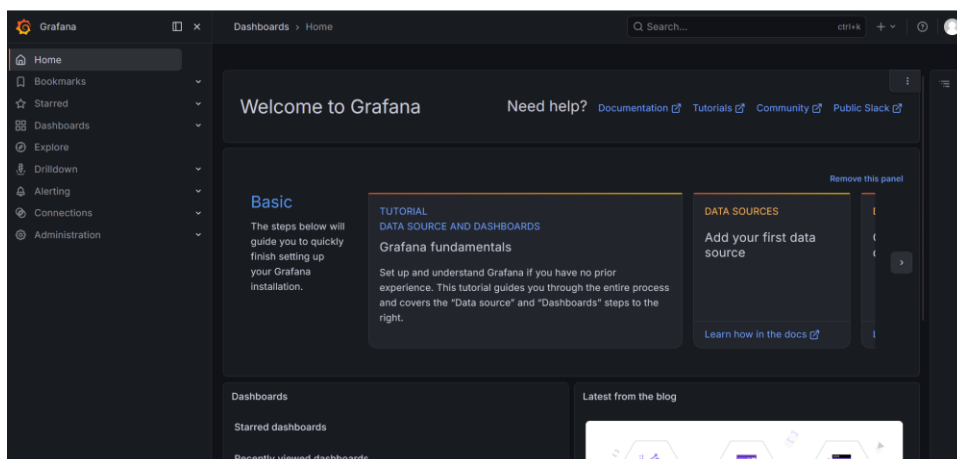


Figure 3.4: Grafana home screen after first launch.

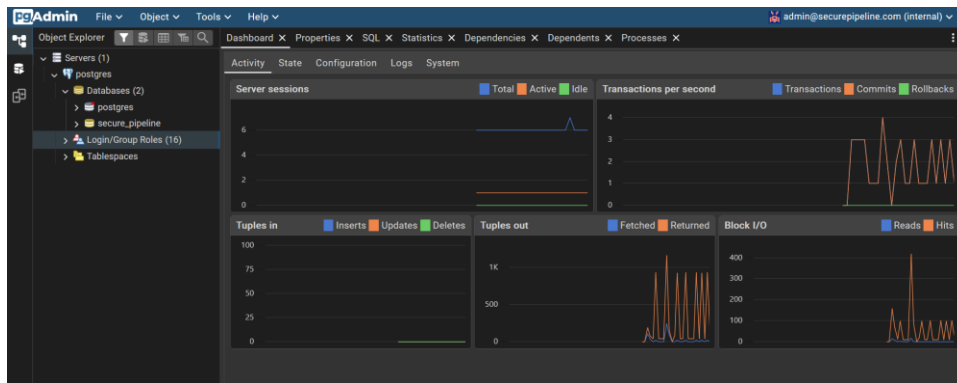
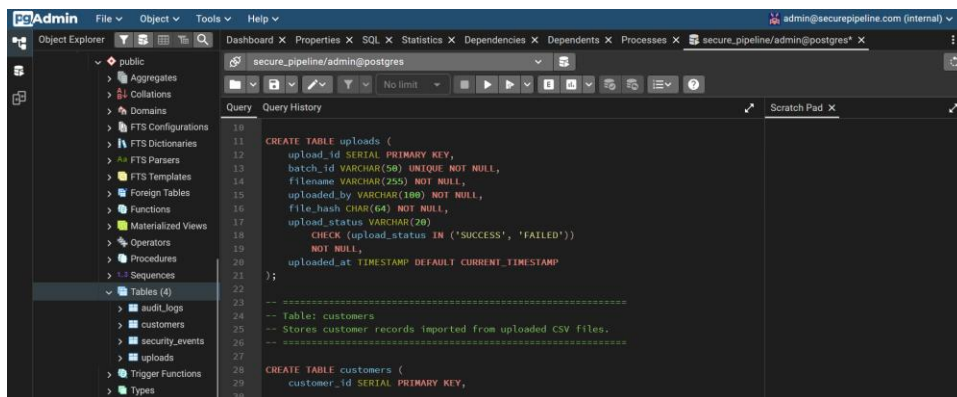


Figure 3.5: pgAdmin connected to the PostgreSQL server, showing live session/transaction activity.

3.2 Database Design

A PostgreSQL database named `secure_pipeline` holds five tables: `users` (accounts and bcrypt password hashes), `uploads` (metadata for every imported dataset, including its SHA-256 hash and status), `customers` (imported records, with sensitive fields encrypted), `audit_logs` (every action performed in the system), and `security_events` (a filtered view of failures, warnings, and permission denials). Full DDL is provided in `schema.sql` (Appendix A).

Figure 3.6: pgAdmin showing the uploads table DDL and the four core tables (`audit_logs`, `customers`, `security_events`, `uploads`).

3.3 Dataset Generation

A synthetic dataset of 100,000 customer records was generated using the Python Faker library (`generate_dataset.py`). Each record includes a first and last name, a unique email address, a unique Kenyan-format phone number, a unique 8-digit national ID, a city drawn from ten Kenyan cities, a sequential account number, a randomly assigned account type (standard, premium, business), and a randomly generated account balance. Uniqueness constraints on email, phone, and national ID are enforced during generation, matching the corresponding UNIQUE constraints in the customers table.

```

> PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python dataset/generate_dataset.py
Generating 100,000 customer records...

Dataset successfully generated.
Records: 100,000
Saved to: dataset/customers.csv
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline>

```

Figure 3.7: `generate_dataset.py` producing 100,000 customer records to `dataset/customers.csv`.

3.4 Authentication

Users authenticate with a username and password. `auth.py` hashes passwords with `bcrypt` at account creation (`hash_password()`) and verifies them at login (`verify_password()`) using `bcrypt`'s constant-time comparison: plaintext passwords are never stored or compared directly. Three roles are supported: ADMIN, ANALYST, and AUDITOR. Every login attempt, successful or failed, is written to `audit_logs` via `logger.py`.

```

pipeline > auth.py > ...
1  import bcrypt
2
3
4  def hash_password(password):
5      """
6      Hash a plaintext password using bcrypt.
7      """
8      password_bytes = password.encode("utf-8")
9
10     hashed = bcrypt.hashpw(

```

```

(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/auth.py
Hashed Password:
$2b$12$5LI.70eC.PNEW0uHU4FCupcqHopQiA.ddtXf6siDz2MXzzO3ql2

Verification:
True

```

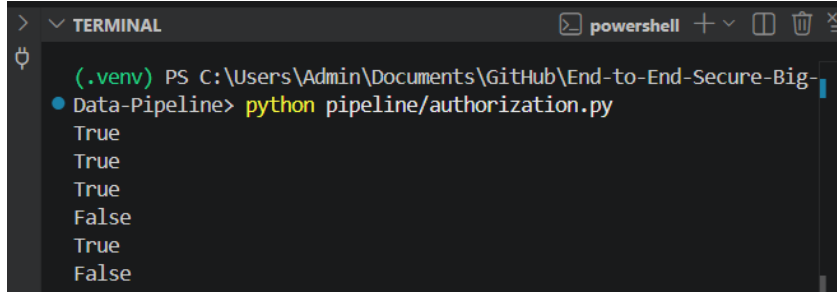
Figure 3.8: `auth.py` hashing a password with `bcrypt` and verifying it (Verification: True).

	user_id [PK] integer	username character varying (50)	password_hash character varying (255)	role character varying (20)	created_at timestamp without time zone
1	1	admin	\$2b\$12\$hb0wuoRFPHEGpoTBzZnsezAps8mmOE6jMbpJkIKm5P52uUthV...	ADMIN	2026-07-16 07:17:56.309778
2	2	analyst	\$2b\$12\$B7aTkvwrm1Vue64JK2wRl6e2MUCzCDYL3RYKWecRitgIKmNtlzQf...	ANALYST	2026-07-16 07:17:56.637593
3	3	auditor	\$2b\$12\$vcKSTAmM/5sWHUJYssDO/iupUw4xK107WYKsndcfPw0xkkI9nOU...	AUDITOR	2026-07-16 07:17:56.90581

Figure 3.9: the `users` table showing `bcrypt` hashes (never plaintext) for admin, analyst, and auditor.

3.5 Role-Based Access Control (RBAC)

Every sensitive action is gated by `authorization.py`'s `has_permission(role, action)` check before it executes. Administrators can validate, import, encrypt, verify integrity, export, and view both audit logs and security events. Analysts can validate datasets and verify integrity only. Auditors have read-only visibility: verify integrity, view audit logs, and view security events. Unauthorized attempts are denied and logged, following the Principle of Least Privilege.

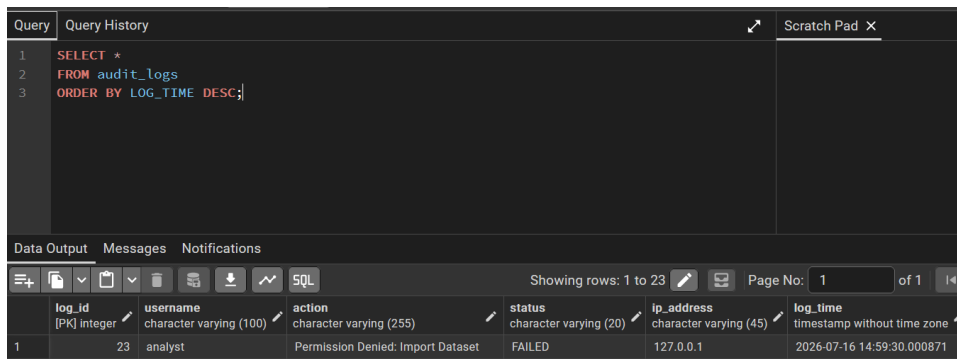


```

> ▼ TERMINAL powershell
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/authorization.py
True
True
True
False
True
False

```

Figure 3.10: `authorization.py`'s `has_permission()` checks returning `True` or `False` for each role and action combination.



Query History

```

1 SELECT *
2 FROM audit_logs
3 ORDER BY LOG_TIME DESC;

```

Data Output

	log_id	username	action	status	ip_address	log_time
	[PK] integer	character varying (100)	character varying (255)	character varying (20)	character varying (45)	timestamp without time zone
1	23	analyst	Permission Denied: Import Dataset	FAILED	127.0.0.1	2026-07-16 14:59:30.000871

Showing rows: 1 to 23 | Page No: 1 of 1

Figure 3.11: an Analyst's attempt to import a dataset denied by RBAC and recorded in `audit_logs`.

3.6 Dataset Validation

Before any data enters the database, `validator.py` checks that the file exists and is a readable CSV, that it is not empty, that all required columns are present, and that there are no missing values, duplicate emails, duplicate account numbers, invalid account types, or negative balances. A failed validation halts the pipeline immediately and is logged.

```

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
-----

Choose an option: 1
=====
DATASET VALIDATION REPORT
=====
Rows: 100,000
Columns: 9

Validation PASSED

Dataset validation completed successfully.

```

Figure 3.12: successful validation: 100,000 rows, 9 columns, PASSED.

```

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
-----

Choose an option: 1
=====
DATASET VALIDATION REPORT
=====
Rows: 100,000
Columns: 8

Validation FAILED

```

Figure 3.13: validation correctly failing against a dataset missing a required column (8 of 9 columns present).

3.7 SHA-256 Hash Generation

Once validation succeeds, hasher.py computes a SHA-256 digest of the dataset and stores it both as a standalone hash file and in the uploads table, providing a trusted reference for later integrity verification.

```

PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/hashe.py
Generated Hash:
076ef1c05acb3b4a0def8feaf81910523548520bb912f482abce2bc55b364e4

Hash saved successfully.
Integrity Check: PASSED
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline>

```

Figure 3.14: *hashe.py* generating and saving a SHA-256 hash, with an immediate integrity self-check passing.

3.8 Secure Import Pipeline

The import workflow chains the previous steps together: validate, hash, save the hash, generate a unique batch ID, encrypt sensitive fields (first name, last name, email, phone, national ID) using Fernet symmetric encryption, insert upload metadata, bulk-insert customer records, and log the outcome. The encryption key is stored at `keys/secret.key`.

```

(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
-----
Choose an option: 2
=====
DATASET VALIDATION REPORT
=====
Rows: 100,000
Columns: 9

Validation PASSED

Dataset validation completed successfully.

Generating SHA-256 hash...
Hash generated successfully.
2439dd3bc2b6c4046c2c09e5c5da3e2c2c6608ae79adadf767b4f1e76537d786

Hash saved to: hashes/customers.sha256

Generated Batch ID: BATCH-5fc4cf27

Importing dataset into PostgreSQL...
Successfully imported 100000 customer records.
Upload ID : 1
Batch ID  : BATCH-5fc4cf27

Pipeline completed successfully.

```

Figure 3.15: a complete pipeline run: validation passes, hash generated, batch ID assigned, 100,000 records imported successfully.

upload_id	batch_id	filename	uploaded_by	file_hash	upload_status
1	BATCH-1fed8112	customers.csv	Beethoven	076ef1c05acb3b4a0def8feaf81910523548520bb912f482abce2bc55b364e4	SUCCESS

Figure 3.16: the resulting row in the uploads table, confirming batch ID, filename, hash, and SUCCESS status.

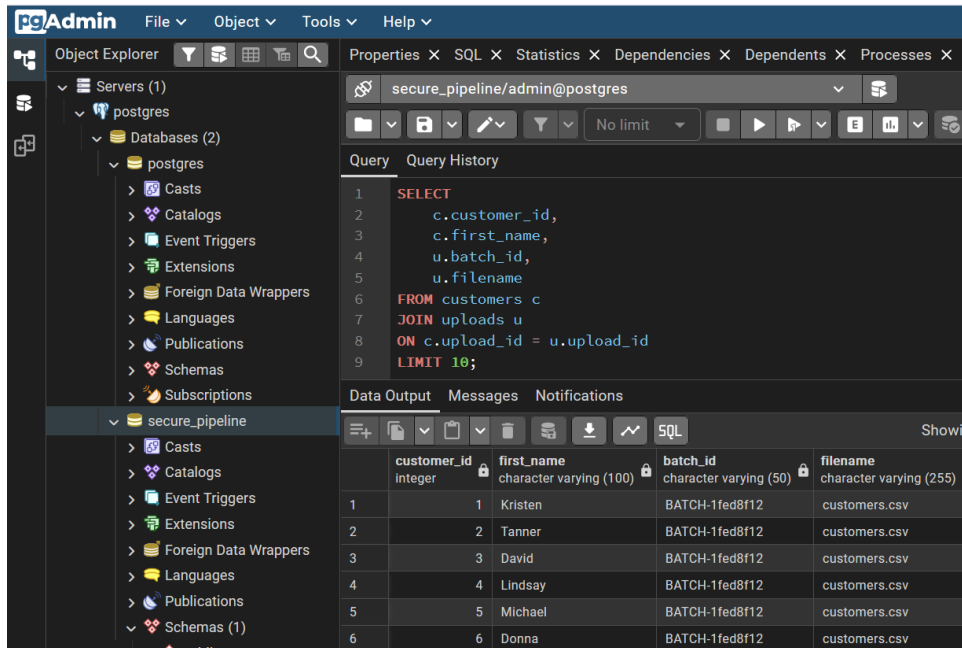


Figure 3.17: a join query across customers and uploads confirming the foreign-key linkage between imported records and their originating batch.

3.9 Audit Logging

Every meaningful action, including logins (successful and failed), validation, hashing, encryption, imports, exports, permission denials, integrity checks, and log views, is written to `audit_logs` with the acting username, action, status, IP address, and timestamp, giving a complete audit trail of system activity.

The screenshot shows the pgAdmin interface. The main pane displays a SQL query and its results. The query is:

```

1 SELECT *
2 FROM audit_logs
3 ORDER BY log_time DESC
4 LIMIT 16;

```

The results table has the following columns: log_id (integer), username (character varying (100)), action (character varying (255)), status (character varying (20)), ip_address (character varying (45)), and log_time (timestamp without time zone). The results show 16 rows of data:

log_id	username	action	status	ip_address	log_time
79	admin	Dataset Integrity Verification	SUCCESS	127.0.0.1	2026-07-16 22:24:18.299182
78	admin	User Login	SUCCESS	127.0.0.1	2026-07-16 22:24:08.031786
77	admin	Dataset Imported	FAILED	127.0.0.1	2026-07-16 22:15:32.561493
76	admin	Sensitive Data Encrypted	SUCCESS	127.0.0.1	2026-07-16 22:08:27.859342
75	admin	SHA-256 Hash Generated	SUCCESS	127.0.0.1	2026-07-16 22:08:24.414935
74	admin	Dataset Validation	SUCCESS	127.0.0.1	2026-07-16 22:08:22.972683
73	admin	Viewed Audit Logs	SUCCESS	127.0.0.1	2026-07-16 22:00:40.632253
72	admin	User Login	SUCCESS	127.0.0.1	2026-07-16 22:00:32.494953
71	analyst	User Logout	SUCCESS	127.0.0.1	2026-07-16 21:59:26.633829
70	analyst	Permission Denied: View Security Ev...	FAILED	127.0.0.1	2026-07-16 21:56:27.727754
69	analyst	User Login	SUCCESS	127.0.0.1	2026-07-16 21:56:02.600527
68	admin	User Logout	SUCCESS	127.0.0.1	2026-07-16 21:55:03.074866
67	admin	Viewed Security Events	SUCCESS	127.0.0.1	2026-07-16 21:49:53.622306
66	admin	Viewed Audit Logs	SUCCESS	127.0.0.1	2026-07-16 21:42:08.954799

Figure 3.18: a representative slice of `audit_logs` spanning logins, logouts, validation, hashing, encryption, a failed import, and integrity checks across two users.

3.10 Dataset Integrity Verification

`verifier.py` recomputes the SHA-256 hash of the current dataset and compares it against the stored hash. A match confirms the dataset has not been altered since import; a mismatch is flagged immediately as a security event.

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Login successful.
Welcome admin.
Role: ADMIN

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
-----
Choose an option: 3

Dataset integrity verified.
The dataset has NOT been modified.
```

Figure 3.19: integrity verification passing: dataset unmodified.

```
=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
-----
Choose an option: 3

WARNING!
Dataset integrity verification FAILED.
The dataset has been modified.
```

Figure 3.20: integrity verification correctly detecting tampering after the dataset was modified.

3.11 Secure Data Export

Administrators can export a specified number of customer records to CSV. Exports at or above a 50,000-record threshold are still allowed to complete, but are flagged with a WARNING and logged as a security event: mirroring Data Loss Prevention (DLP) monitoring, where unusually large exports are surfaced for review without necessarily blocking legitimate operations.

```

(.venv) PS C:\Users\Admin\Documents\Github\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Login successful.
Welcome admin.
Role: ADMIN

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN

-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Export Customer Data
7. Exit
-----

Choose an option: 6

Enter number of records to export: 2000
C:\Users\Admin\Documents\Github\End-to-End-Secure-Big-Data-Pipeline\pipeline\exporter.py:61: UserWarning: pandas
s only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection.
Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df = pd.read_sql(query, connection, params=(rows,))

Export completed successfully.
Records exported: 2,000
File saved to: exports/customers_export.csv

```

Figure 3.21: a normal export (2,000 records) completing with no warning.

```

(.venv) PS C:\Users\Admin\Documents\Github\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
6. Export Customer Data
7. Exit

-----
Choose an option: 6

Enter number of records to export: 60000
C:\Users\Admin\Documents\Github\End-to-End-Secure-Big-Data-Pipeline\pipeline\exporter.py:61: UserWarning: pandas only supports SQLAlchemy connectable (engine
/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df = pd.read_sql(query, connection, params=(rows,))

WARNING: Large data export detected.

Export completed successfully.
Records exported: 60,000
File saved to: exports/customers_export.csv

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN

-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Export Customer Data
7. Exit
-----

Choose an option: 5

=====
LATEST SECURITY EVENTS
=====
ID  USERNAME  ACTION              STATUS  TIME
---  -
116 admin     Bulk data Export Detected  WARNING  2026-07-17 00:24:56.586742

```

Figure 3.22: a bulk export (60,000 records) correctly triggering a WARNING and a logged security event.

3.12 Security Monitoring (Grafana)

Grafana is connected directly to PostgreSQL and visualizes both operational and security data: successful and failed logins, successful imports, integrity results, audit log activity, warnings, and security event trends, refreshing automatically for near real-time monitoring.

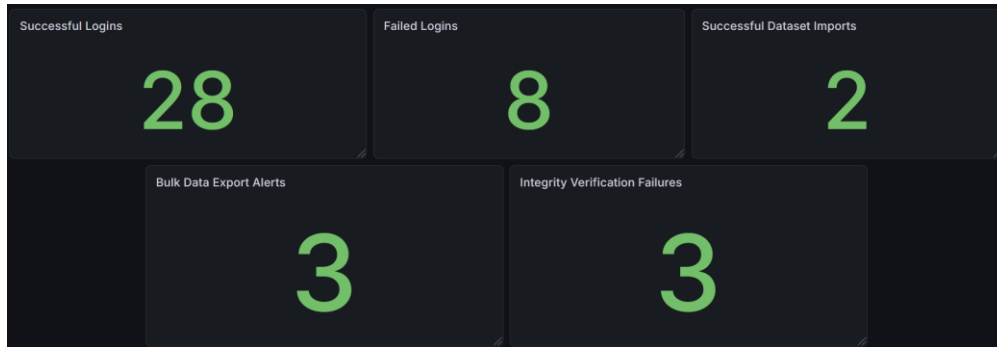


Figure 3.23: Grafana summary cards: successful/failed logins, successful imports, bulk export alerts, and integrity failures.

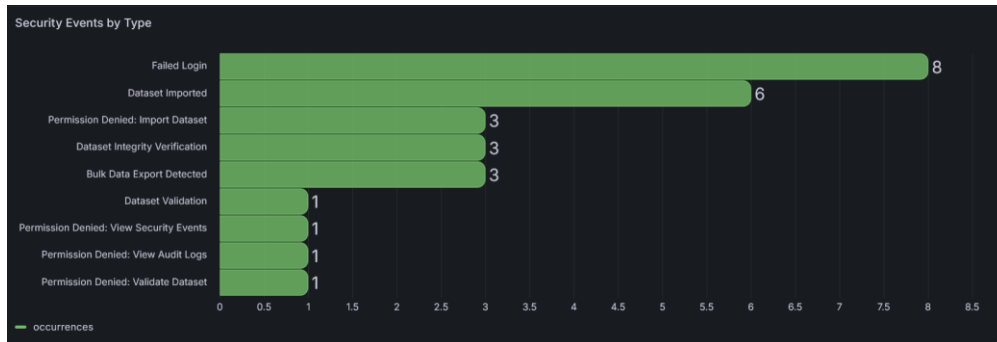


Figure 3.24: Grafana breakdown of security events by type.



Figure 3.25: Grafana pipeline operations summary (validations, hashes, encryptions, exports).

3.13 Desktop Graphical User Interface

In addition to the command-line interface, a full desktop GUI was implemented using CustomTkinter (app.py). It reuses the existing backend modules directly, so the business logic remains unchanged, while adding a native file picker for dataset uploads, a role-aware sidebar that shows or hides screens based on the logged-in user's actual permissions, live progress feedback during import, and a one-click link to the Grafana monitoring dashboard. This closes out the GUI originally planned as a future improvement.

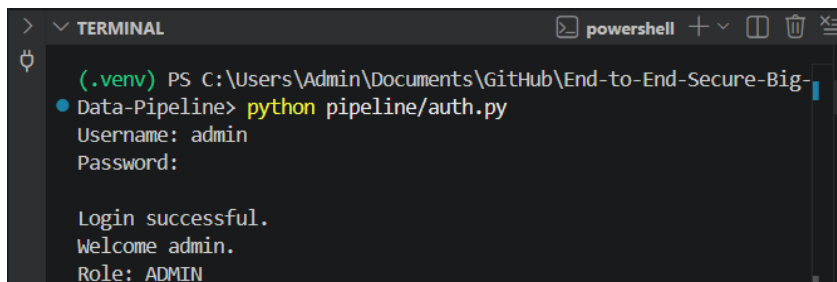
4. Security Testing Matrix

Nine security tests were executed against the running system. Each test below records its objective, procedure, expected result, actual result, and supporting evidence captured directly from the terminal, pgAdmin, or the desktop GUI.

Test 1: Authentication

Objective	Confirm that valid credentials are accepted and invalid credentials are rejected, with both outcomes logged.
Procedure	Log in via the CLI as admin with the correct password, then attempt to log in as analyst with an incorrect password.
Expected Result	Correct credentials succeed and return the user's role; incorrect credentials are rejected with no information disclosed about which field was wrong; both attempts are written to audit_logs.
Actual Result	admin logged in successfully and was shown Role: ADMIN. analyst's incorrect login was rejected with "Invalid username or password. Access denied." Both events (User Login / SUCCESS and Failed Login / FAILED) appear in audit_logs.

Result: PASS

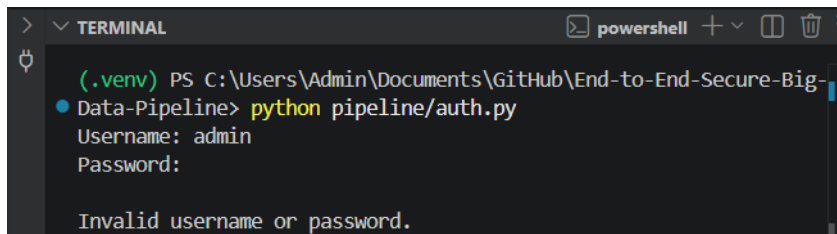


```

> v TERMINAL powershell + v [ ] [X]
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/auth.py
Username: admin
Password:

Login successful.
Welcome admin.
Role: ADMIN
  
```

Figure 4.1: successful admin login.



```

> v TERMINAL powershell + v [ ] [X]
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/auth.py
Username: admin
Password:

Invalid username or password.
  
```

Figure 4.2: rejected login with incorrect credentials.

LATEST AUDIT LOGS				
ID	USERNAME	ACTION	STATUS	TIME
65	admin	User Login	SUCCESS	2026-07-16 21:41:39.669937
64	analyst	Failed Login	FAILED	2026-07-16 21:39:10.830014

Figure 4.3: both events recorded in audit_logs (SUCCESS and FAILED).

Test 2: Role-Based Access Control (RBAC)

Objective	Confirm that each role can only perform the actions authorization.py grants it, and that disallowed actions are denied and logged.
Procedure	Log in as analyst and attempt to run the Import Dataset action (Analyst permissions are validate + verify_hash only). Separately, confirm Admin can access View Security Events, which Analyst cannot.

Expected Result	Analyst's import attempt is denied before any data is touched; the denial is logged as a security event. Admin's access to Security Events succeeds.
Actual Result	analyst's import attempt returned "Permission Denied: Import Dataset" and was logged as FAILED (audit_logs ID 23 / 41). admin successfully viewed the Security Events screen.

Result: PASS

log_id	username	action	status	ip_address	log_time
23	analyst	Permission Denied: Import Dataset	FAILED	127.0.0.1	2026-07-16 14:59:30.000871

Figure 4.4: analyst's import attempt denied by RBAC and logged.

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/authorization.py
True
True
True
False
True
False
```

Figure 4.5: authorization.py's has_permission() checks returning True/False per role/action combination.

```

=====
Logged in as : admin
Role         : ADMIN
=====
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
=====
Choose an option: 5
=====
LATEST SECURITY EVENTS
=====

```

ID	USERNAME	ACTION	STATUS	TIME
64	analyst	Failed Login	FAILED	2026-07-16 21:39:10.830014
57	admin	Failed Login	FAILED	2026-07-16 19:41:31.977232
55	admin	Dataset Integrity Verification	FAILED	2026-07-16 17:26:31.074328
52	auditor	Dataset Integrity Verification	SUCCESS	2026-07-16 17:13:56.188709
50	auditor	Failed Login	FAILED	2026-07-16 17:13:20.293281
49	auditor	Failed Login	FAILED	2026-07-16 17:12:58.827141
46	auditor	Permission Denied: Import Dataset	FAILED	2026-07-16 16:55:31.441319
45	auditor	Permission Denied: Validate Dataset	FAILED	2026-07-16 16:55:23.127958
42	analyst	Permission Denied: View Audit Logs	FAILED	2026-07-16 16:54:58.538372
41	analyst	Permission Denied: Import Dataset	FAILED	2026-07-16 16:54:48.961117
34	admin	Sensitive Data Encrypted	SUCCESS	2026-07-16 16:52:02.467779
29	admin	Failed Login	FAILED	2026-07-16 16:51:17.890718
27	admin	Sensitive Data Encrypted	SUCCESS	2026-07-16 15:36:56.831626
23	analyst	Permission Denied: Import Dataset	FAILED	2026-07-16 14:59:30.000871
18	auditor	Failed Login	FAILED	2026-07-16 14:32:10.076342

Figure 4.6: admin successfully viewing Security Events (permission granted).

Test 3: Sensitive Data Encryption

Objective	Confirm that first name, last name, email, phone, and national ID are encrypted before storage, and can be correctly decrypted.
Procedure	Run encryption.py's encrypt/decrypt round trip on a sample value, then query the customers table directly in pgAdmin to inspect stored values.
Expected Result	The round-trip test returns the original plaintext after decryption. Querying customers table shows ciphertext, not plaintext, for all five sensitive columns.
Actual Result	encrypt_data()/decrypt_data() correctly round-tripped a sample national ID (12345678 → ciphertext → 12345678). Querying the live customers table confirmed first_name, last_name, email, phone, and national_id are all stored as Fernet ciphertext (gAAAAABq...).

Result: PASS

```

Data-Pipeline> python pipeline/encryption.py
Original:
12345678

Encrypted:
gAAAAABqWkdI5lX92WRNVm0ZpZvoAkvpvMNIlf0dhuBRsutytfGMe12amU1YsA
wjgbmkV0ou9RMkwoJU_67KeCyDx00-5ow==

Decrypted:
12345678

```

Figure 4.7: encrypt/decrypt round trip returning the original value.

	customer_id (PK) integer	upload integer	first_name text	last_name text	email text	phone text	national_id text	city character varying (20)	account_number character varying (20)	account_type text
2	2	1	gAAAAABqW...	gAAAAABqWOSxbK...	gAAAAABqWOSxh...	gAAAAABqWOSxF7...	gAAAAABqWOSxhH...	Nairobi	ACC000002	standa...
3	3	1	gAAAAABqW...	gAAAAABqWOSx04...	gAAAAABqWOSx8...	gAAAAABqWOSxof...	gAAAAABqWOSxko...	Kitale	ACC000003	standa...
4	4	1	gAAAAABqW...	gAAAAABqWOSx9...	gAAAAABqWOSxK...	gAAAAABqWOSxe2...	gAAAAABqWOSxN...	Kisumu	ACC000004	standa...
5	5	1	gAAAAABqW...	gAAAAABqWOSxlo...	gAAAAABqWOSxE...	gAAAAABqWOSxSv...	gAAAAABqWOSxA3...	Meru	ACC000005	standa...
6	6	1	gAAAAABqW...	gAAAAABqWOSxvJ...	gAAAAABqWOSxX...	gAAAAABqWOSxb...	gAAAAABqWOSx_O...	Nakuru	ACC000006	premiu...
7	7	1	gAAAAABqW...	gAAAAABqWOSxi9...	gAAAAABqWOSxy...	gAAAAABqWOSxib...	gAAAAABqWOSxC...	Kisumu	ACC000007	standa...
8	8	1	gAAAAABqW...	gAAAAABqWOSx0K...	gAAAAABqWOSxiz...	gAAAAABqWOSxZz...	gAAAAABqWOSxIIG...	Kitale	ACC000008	busine...
9	9	1	gAAAAABqW...	gAAAAABqWOSxXZ...	gAAAAABqWOSxbl...	gAAAAABqWOSxO...	gAAAAABqWOSxW...	Kitale	ACC000009	busine...
10	10	1	gAAAAABqW...	gAAAAABqWOSxhi...	gAAAAABqWOSxY...	gAAAAABqWOSxv...	gAAAAABqWOSx8k...	Meru	ACC000010	standa...

Figure 4.8: customers table in pgAdmin showing all sensitive columns as ciphertext.

Test 4: Dataset Integrity Verification

Objective	Confirm that an unmodified dataset passes integrity verification, and a modified dataset is correctly flagged.
Procedure	Run Verify Dataset Integrity against the original, unmodified customers.csv, then repeat after manually editing the file.
Expected Result	Unmodified dataset: hashes match, verification passes. Modified dataset: hashes differ, verification fails with a warning, and the event is logged.
Actual Result	Unmodified run returned "Dataset integrity verified. The dataset has NOT been modified." After modification, the same check returned "WARNING! Dataset integrity verification FAILED. The dataset has been modified." Both outcomes were logged accordingly.

Result: PASS

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
> TERMINAL
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : auditor
Role        : AUDITOR
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. Exit
-----

Choose an option: 3

Dataset integrity verified.
The dataset has NOT been modified.

```

Figure 4.9: integrity check passing on an unmodified dataset.

```

(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role        : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. Exit
-----

Choose an option: 3

WARNING!
Dataset integrity verification FAILED.
The dataset has been modified.

```

Figure 4.10: integrity check correctly detecting tampering.

Test 5: Audit Logging

Objective	Confirm that logins, validations, hashing, encryption, imports, exports, and permission denials are all captured in audit_logs with correct status and timestamp.
Procedure	Perform a representative session (login, validate, hash, encrypt, import) across two different user accounts, then query audit_logs directly in pgAdmin.
Expected Result	Every action performed generates a corresponding row in audit_logs with the correct username, action, status, and timestamp, in chronological order.
Actual Result	audit_logs contained a complete, correctly ordered trail: User Login, Dataset Validation, SHA-256 Hash Generated, Sensitive Data Encrypted, and Dataset Imported entries for both admin and analyst sessions, including one deliberately FAILED import.

Result: **PASS**

	log_id [PK] integer	username character varying (100)	action character varying (255)	status character varying (20)	ip_address character varying (45)	log_time timestamp without time zone
1	1	admin	User Login	SUCCESS	127.0.0.1	2026-07-16 12:38:44.007005
2	2	admin	Failed Login	FAILED	127.0.0.1	2026-07-16 12:47:52.250814
3	3	admin	User Login	SUCCESS	127.0.0.1	2026-07-16 13:20:26.643541
4	4	admin	Dataset Imported	FAILED	127.0.0.1	2026-07-16 13:21:29.398082
5	5	admin	User Login	SUCCESS	127.0.0.1	2026-07-16 13:38:13.15488
6	6	admin	Dataset Validation	SUCCESS	127.0.0.1	2026-07-16 13:38:14.621782
7	7	admin	SHA-256 Hash Generat...	SUCCESS	127.0.0.1	2026-07-16 13:38:16.778761
8	8	admin	Dataset Imported	FAILED	127.0.0.1	2026-07-16 13:39:27.591941
9	9	analyst	User Login	SUCCESS	127.0.0.1	2026-07-16 14:03:29.036967
10	10	analyst	Dataset Validation	SUCCESS	127.0.0.1	2026-07-16 14:03:29.488755
11	11	analyst	SHA-256 Hash Generat...	SUCCESS	127.0.0.1	2026-07-16 14:03:29.547978
12	12	analyst	Sensitive Data Encrypted	SUCCESS	127.0.0.1	2026-07-16 14:03:29.815106
13	13	analyst	Dataset Imported	SUCCESS	127.0.0.1	2026-07-16 14:05:40.327638

Figure 4.11: audit_logs trail across two users, showing the full validate→hash→encrypt→import sequence.

Test 6: Secure End-to-End Pipeline Execution

Objective	Confirm that a single pipeline run correctly chains validation, hashing, batch ID generation, encryption, and import without manual intervention between steps.
Procedure	Select Run Secure Import Pipeline from the menu and allow it to execute against the full 100,000-record dataset.
Expected Result	The pipeline validates, hashes, generates a batch ID, encrypts sensitive fields, imports all records, and reports success: with no manual steps required between stages.
Actual Result	The pipeline validated the dataset (100,000 rows / 9 columns, PASSED), generated and saved a SHA-256 hash, created Batch ID BATCH-5fc4cf27, imported 100,000 customer records, and reported "Pipeline completed successfully."

Result: PASS

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py

Generating SHA-256 hash...
Hash generated successfully.
076ef1c05acb3b4a0def8feaaaf81910523548520bb912f482abce2bc55b364e4

Hash saved to: hashes/customers.sha256

Generated Batch ID: BATCH-1fed8f12

Importing dataset into PostgreSQL...
Successfully imported 100000 customer records.
Upload ID : 1
Batch ID : BATCH-1fed8f12

Pipeline completed successfully.
```

Figure 4.12: a full pipeline run from validation through successful import.

Test 7: Dataset Validation

Objective	Confirm that a well-formed dataset passes validation, and a malformed dataset (missing a required column) is correctly rejected.
Procedure	Run Validate Dataset against the standard 9-column dataset, then repeat against a copy with one required column removed.

Expected Result	The 9-column dataset passes; the 8-column dataset is rejected before it can proceed further in the pipeline.
Actual Result	The full dataset returned Rows: 100,000, Columns: 9, and Validation PASSED. The modified dataset returned Rows: 100,000, Columns: 8, and Validation FAILED, correctly halting the pipeline.

Result: PASS

```

TERMINAL
PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
=====
DATASET VALIDATION REPORT
=====
Rows: 100,000
Columns: 9

Validation PASSED

Pipeline can continue.
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline>

```

Figure 4.13: validation passing on the correctly structured dataset.

```

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
-----
Choose an option: 1
=====
DATASET VALIDATION REPORT
=====
Rows: 100,000
Columns: 8

Validation FAILED

```

Figure 4.14: validation correctly failing on a dataset missing a required column.

Test 8: Bulk Data Export Detection

Objective	Confirm exports below the 50,000-record threshold complete silently, while exports at or above the threshold are flagged as a security event without blocking the export.
Procedure	Export 2,000 records (below threshold), then export 60,000 records (above threshold).
Expected Result	The 2,000-record export completes with no warning. The 60,000-record export completes successfully but triggers a WARNING and is logged as a Bulk Data Export Detected security event.

Actual Result	The 2,000-record export completed silently. The 60,000-record export printed "WARNING: Large data export detected," completed successfully, and was logged as a security event (see Figures 3.21 and 3.22 for full terminal output).
----------------------	--

Result: PASS

```

(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
6. Export Customer Data
7. Exit

-----
Choose an option: 6

Enter number of records to export: 60000
C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline\pipeline\exporter.py:61: UserWarning: pandas only supports SQLAlchemy connectable (engine
/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df = pd.read_sql(query, connection, params=(rows,))

WARNING: Large data export detected.

Export completed successfully.
Records exported: 60,000
File saved to: exports/customers_export.csv

-----
END-TO-END SECURE BIG DATA PIPELINE
-----
Logged in as : admin
Role       : ADMIN
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Export Customer Data
7. Exit
-----
Choose an option: 5

-----
LATEST SECURITY EVENTS
-----
ID  USERNAME  ACTION                                STATUS  TIME
-----
116 admin    Bulk Data Export Detected              WARNING 2026-07-17 08:24:56.586742

```

Figure 4.15: a 60,000-record export triggering the bulk-export warning and security event.

Test 9: Security Events Monitoring

Objective	Confirm that the Security Events view surfaces only security-relevant activity (failures, warnings, permission denials, integrity issues) separately from the general audit log, and that access to it is itself governed by RBAC.
Procedure	Log in as auditor (who has view_security_events permission) and select View Security Events.
Expected Result	A filtered view is returned showing only FAILED/WARNING entries and permission-denial/integrity/encryption events: not routine successful activity.
Actual Result	The Security Events screen returned a filtered list including Bulk Data Export Detected (WARNING), Dataset Validation (FAILED), Sensitive Data Encrypted (SUCCESS, shown because it matches the encryption filter), Dataset Imported (FAILED), and Dataset Integrity Verification (mixed): correctly separated from routine audit log noise.

Result: PASS

```

(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Username: auditor
Password:

Login successful.
Welcome auditor.
Role: AUDITOR

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : auditor
Role       : AUDITOR
-----
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Export Customer Data
7. Exit
-----
Choose an option: 5

=====
LATEST SECURITY EVENTS
=====

```

ID	USERNAME	ACTION	STATUS	TIME
116	admin	Bulk Data Export Detected	WARNING	2026-07-17 08:24:56.586742
112	admin	Bulk Data Export Detected	WARNING	2026-07-17 08:22:51.186813
107	admin	Bulk Data Export Detected	WARNING	2026-07-17 08:14:23.765772
96	admin	Dataset Validation	FAILED	2026-07-16 23:10:02.649189
91	admin	Sensitive Data Encrypted	SUCCESS	2026-07-16 22:45:06.299602
87	admin	Dataset Imported	FAILED	2026-07-16 22:40:06.703560
86	admin	Sensitive Data Encrypted	SUCCESS	2026-07-16 22:39:05.154421
81	admin	Dataset Integrity Verification	FAILED	2026-07-16 22:28:06.939026
80	admin	Dataset Integrity Verification	FAILED	2026-07-16 22:27:51.775901
79	admin	Dataset Integrity Verification	SUCCESS	2026-07-16 22:24:18.299182
77	admin	Dataset Imported	FAILED	2026-07-16 22:15:32.561493

Figure 4.16: auditor's filtered Security Events view.

5. Simulated Security Incidents

To demonstrate that the implemented controls detect and respond to real security incidents, not just pass isolated unit tests, three incidents were simulated directly against the desktop GUI, which enforces the same authentication, RBAC, and integrity logic as the CLI.

Incident 1: Unauthorized Login Attempt

A login was attempted using a valid username (analyst) with an incorrect or blank password.

Detection & response: the GUI rejected the attempt inline with "Invalid username or password," without revealing whether the username or password was the incorrect field. The attempt was logged as a Failed Login event.

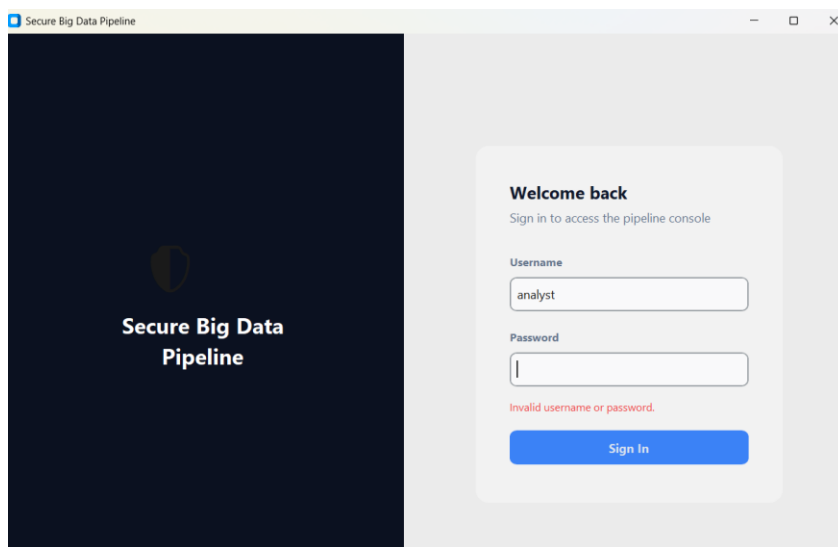


Figure 5.1: GUI correctly rejecting an unauthorized login attempt.

Incident 2: Privilege Escalation Attempt

An authenticated Analyst attempted to perform an action outside their granted permissions (importing a dataset), which is reserved for Admin under the RBAC policy.

Detection & response: the action was blocked before touching any data, and the denial was recorded as a security event ("Permission Denied: Import Dataset", log #23). The Auditor's own permission grid confirms the boundary is enforced consistently: the same Verify/Audit Logs/Security Events access is granted, while Validate, Import, Encrypt, and Export remain blocked.

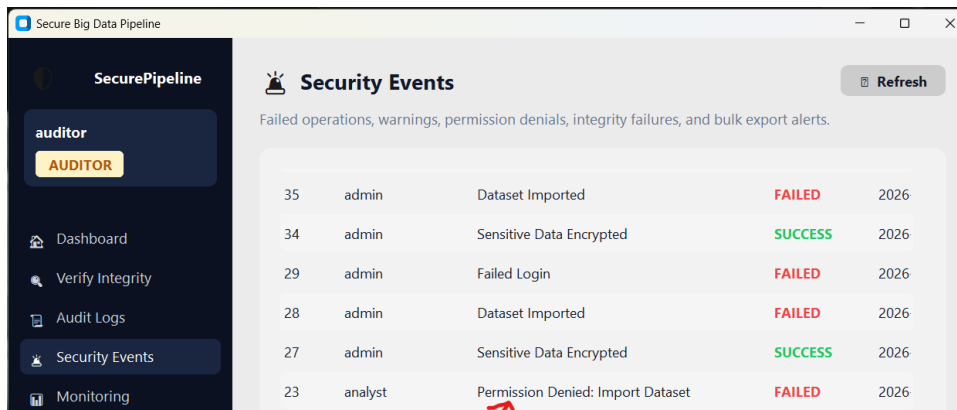


Figure 5.2: Security Events screen showing the blocked privilege escalation attempt (analyst → Permission Denied: Import Dataset).

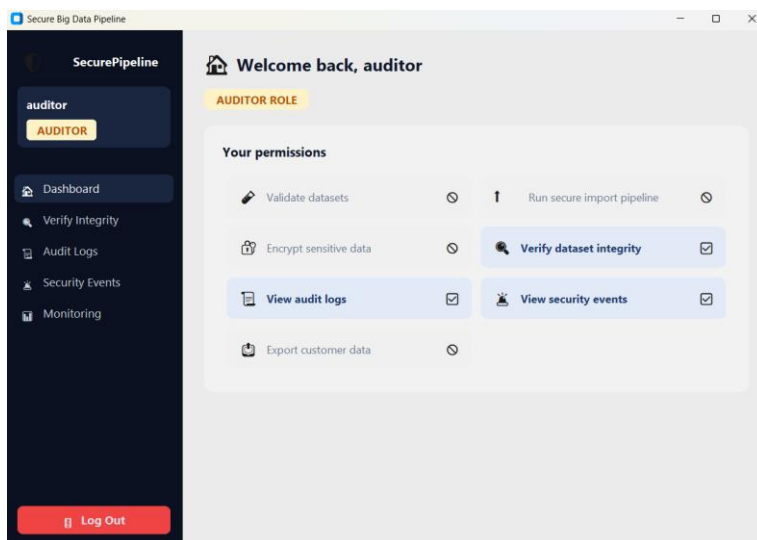


Figure 5.3: Auditor's permission dashboard confirming only Verify Integrity, Audit Logs, and Security Events are granted.

Incident 3: Dataset Tampering

The customer dataset was modified after import, then checked against its previously stored SHA-256 hash using the Verify Integrity screen.

Detection & response: the current hash no longer matched the stored hash, and the GUI immediately surfaced "WARNING: dataset has been modified since import." This is the same integrity-verification logic exercised in Test 4, now demonstrated end-to-end through the GUI.

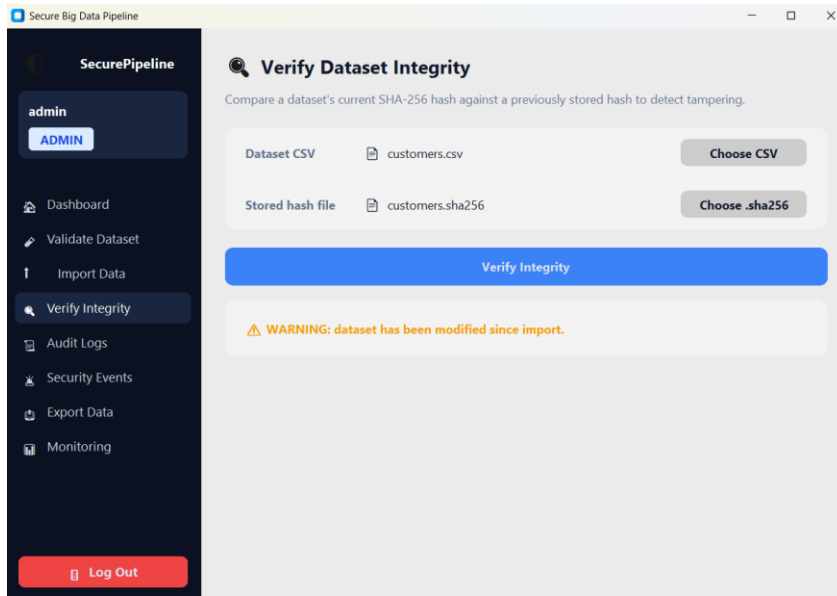


Figure 5.4: GUI detecting and flagging dataset tampering.

Summary

- All three incidents were detected using the existing controls: no additional tooling was required.
- Each incident produced a corresponding, correctly labeled entry in the audit trail.
- RBAC and integrity verification behaved identically whether triggered from the CLI or the GUI, confirming the two front ends share one consistent security boundary.

6. Evidence Portfolio

This section collects supplementary screenshots, logs, and dashboard captures that support the testing matrix and implementation report but were not already shown inline. Raw screenshot files, configuration files, scripts, and the dataset are also submitted alongside this report as supporting evidence (see Appendix A).

6.1 Environment & Connectivity

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/test_connection.py
Connected successfully!
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> 
```

Figure 6.1: `test_connection.py` confirming successful connectivity to PostgreSQL.

6.2 Additional Authentication Evidence

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Username: admin
Password:

Login successful.
Welcome admin.
Role: ADMIN
```

Figure 6.2: a second successful admin login session.

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Username: analyst
Password:

Invalid username or password.
Access denied.
```

Figure 6.3: a second failed login attempt (analyst, invalid credentials).

6.3 Additional RBAC Evidence

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Username: analyst
Password:

Login successful.
Welcome analyst.
Role: ANALYST

=====
END-TO-END SECURE BIG DATA PIPELINE
=====
Logged in as : analyst
Role       : ANALYST
=====
1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
```

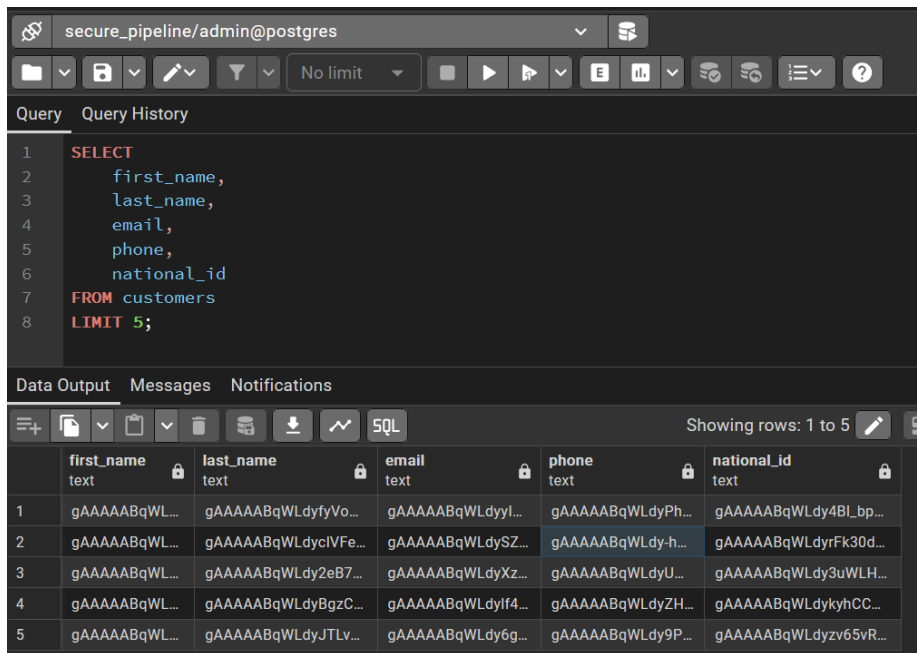
Figure 6.4: analyst successfully authenticating and reaching the role-appropriate menu.

```
70 analyst Permission Denied: View Security Events FAILED 2026-07-16 21:56:27.727754
```

Figure 6.5: a single `audit_logs` row confirming analyst's denied attempt to view Security Events.

6.4 Additional Encryption Evidence

```
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/encryption.py
Encryption key generated successfully.
```

Figure 6.6: *encryption.py* generating the Fernet encryption key.


secure_pipeline/admin@postgres

Query

```

1  SELECT
2      first_name,
3      last_name,
4      email,
5      phone,
6      national_id
7  FROM customers
8  LIMIT 5;

```

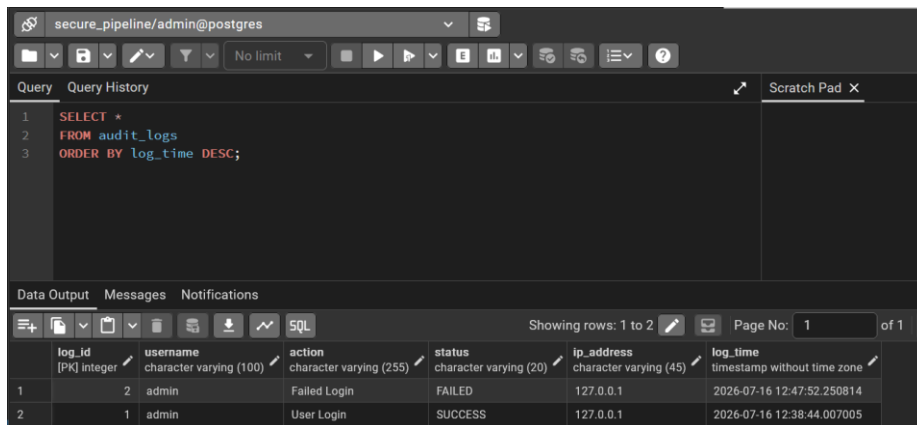
Data Output Messages Notifications

Showing rows: 1 to 5

	first_name text	last_name text	email text	phone text	national_id text
1	gAAAAABqWL...	gAAAAABqWLDyfyVo...	gAAAAABqWLDyyl...	gAAAAABqWLDyPh...	gAAAAABqWLDy4BLbp...
2	gAAAAABqWL...	gAAAAABqWLDyclVFe...	gAAAAABqWLDySZ...	gAAAAABqWLDy-h...	gAAAAABqWLDyFk30d...
3	gAAAAABqWL...	gAAAAABqWLDy2eB7...	gAAAAABqWLDyXz...	gAAAAABqWLDyU...	gAAAAABqWLDy3uWLH...
4	gAAAAABqWL...	gAAAAABqWLDyBgzc...	gAAAAABqWLDyIf4...	gAAAAABqWLDyZH...	gAAAAABqWLDykyhCC...
5	gAAAAABqWL...	gAAAAABqWLDyJTLv...	gAAAAABqWLDy6g...	gAAAAABqWLDy9P...	gAAAAABqWLDyzy65vR...

Figure 6.7: a *pgAdmin* query returning encrypted *first_name*, *last_name*, *email*, *phone*, and *national_id* values for five customer records.

6.5 Additional Audit Logging Evidence



secure_pipeline/admin@postgres

Query

```

1  SELECT *
2  FROM audit_logs
3  ORDER BY log_time DESC;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	log_id [PK] integer	username character varying (100)	action character varying (255)	status character varying (20)	ip_address character varying (45)	log_time timestamp without time zone
1	2	admin	Failed Login	FAILED	127.0.0.1	2026-07-16 12:47:52.250814
2	1	admin	User Login	SUCCESS	127.0.0.1	2026-07-16 12:38:44.007005

Figure 6.8: *audit_logs* isolated to login-related events for a single session.

The screenshot shows a SQL query editor with the following query:

```

1 SELECT
2     log_id,
3     username,
4     action,
5     status,
6     log_time
7 FROM audit_logs
8 ORDER BY log_id DESC;

```

Below the query editor, the 'Data Output' tab is active, displaying a table with 8 rows. The table has the following columns: log_id (PK Integer), username (character varying (100)), action (character varying (255)), status (character varying (20)), and log_time (timestamp without time zone).

log_id	username	action	status	log_time
8	admin	Dataset Imported	FAILED	2026-07-16 13:39:27.591941
7	admin	SHA-256 Hash Generat...	SUCCESS	2026-07-16 13:38:16.778761
6	admin	Dataset Validation	SUCCESS	2026-07-16 13:38:14.621782
5	admin	User Login	SUCCESS	2026-07-16 13:38:13.15488
4	admin	Dataset Imported	FAILED	2026-07-16 13:21:29.398082
3	admin	User Login	SUCCESS	2026-07-16 13:20:26.643541
2	admin	Failed Login	FAILED	2026-07-16 12:47:52.250814
1	admin	User Login	SUCCESS	2026-07-16 12:38:44.007005

Figure 6.9: audit_logs showing the pipeline event trail (validation, hashing, encryption, import) for an admin session.

6.6 Additional Pipeline Execution Evidence

```

TERMINAL
Decrypted:
12345678
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
Username: admin
Password:

Login successful.
Welcome admin.
Role: ADMIN
=====
DATASET VALIDATION REPORT
=====
Rows: 100,000
Columns: 9

Validation PASSED

Pipeline can continue.

Generating SHA-256 hash...
Hash generated successfully.
076ef1c05acb3b4a0def8feaaf81910523548520bb912f482abce2bc55b364e4

Hash saved to: hashes/customers.sha256

Generated Batch ID: BATCH-18918c05

Importing dataset into PostgreSQL...
Successfully imported 100000 customer records.
Upload ID : 1
Batch ID : BATCH-18918c05

Pipeline completed successfully.

```

Figure 6.10: a second full pipeline run, confirming repeatable, consistent behavior across executions.

```
(Ctrl+Shift+M)
(.venv) PS C:\Users\Admin\Documents\GitHub\End-to-End-Secure-Big-Data-Pipeline> python pipeline/main.py
END-TO-END SECURE BIG DATA PIPELINE

=====
Logged in as : admin
Role          : ADMIN
=====

1. Validate Dataset
2. Run Secure Import Pipeline
3. Verify Dataset Integrity
4. View Audit Logs
5. View Security Events
6. Exit
=====

Choose an option: 5

=====
LATEST SECURITY EVENTS
=====
```

ID	USERNAME	ACTION	STATUS	TIME
57	admin	Failed Login	FAILED	2026-07-16 19:41:31.977232
55	admin	Dataset Integrity Verification	FAILED	2026-07-16 17:26:31.074328
52	auditor	Dataset Integrity Verification	SUCCESS	2026-07-16 17:13:56.188709
50	auditor	Failed Login	FAILED	2026-07-16 17:13:20.293281
49	auditor	Failed Login	FAILED	2026-07-16 17:12:58.827141
46	auditor	Permission Denied: Import Dataset	FAILED	2026-07-16 16:55:31.441319
45	auditor	Permission Denied: Validate Dataset	FAILED	2026-07-16 16:55:23.127958
42	analyst	Permission Denied: View Audit Logs	FAILED	2026-07-16 16:54:58.538372
41	analyst	Permission Denied: Import Dataset	FAILED	2026-07-16 16:54:48.961117
34	admin	Sensitive Data Encrypted	SUCCESS	2026-07-16 16:52:02.467779
29	admin	Failed Login	FAILED	2026-07-16 16:51:17.890718
27	admin	Sensitive Data Encrypted	SUCCESS	2026-07-16 15:36:56.831626
23	analyst	Permission Denied: Import Dataset	FAILED	2026-07-16 14:59:30.000871
18	auditor	Failed Login	FAILED	2026-07-16 14:32:10.076342
17	auditor	Failed Login	FAILED	2026-07-16 14:31:32.624013
12	analyst	Sensitive Data Encrypted	SUCCESS	2026-07-16 14:03:29.815106
2	admin	Failed Login	FAILED	2026-07-16 12:47:52.250814

```
=====
Security events displayed successfully.
```

Figure 6.11: Admin's own view of Security Events, for comparison against the Auditor's filtered view shown in Figure 4.16.

6.7 Grafana Monitoring Dashboard: Full Panel Set

The complete set of Grafana panels configured for this project, beyond the three shown in the Implementation Report:

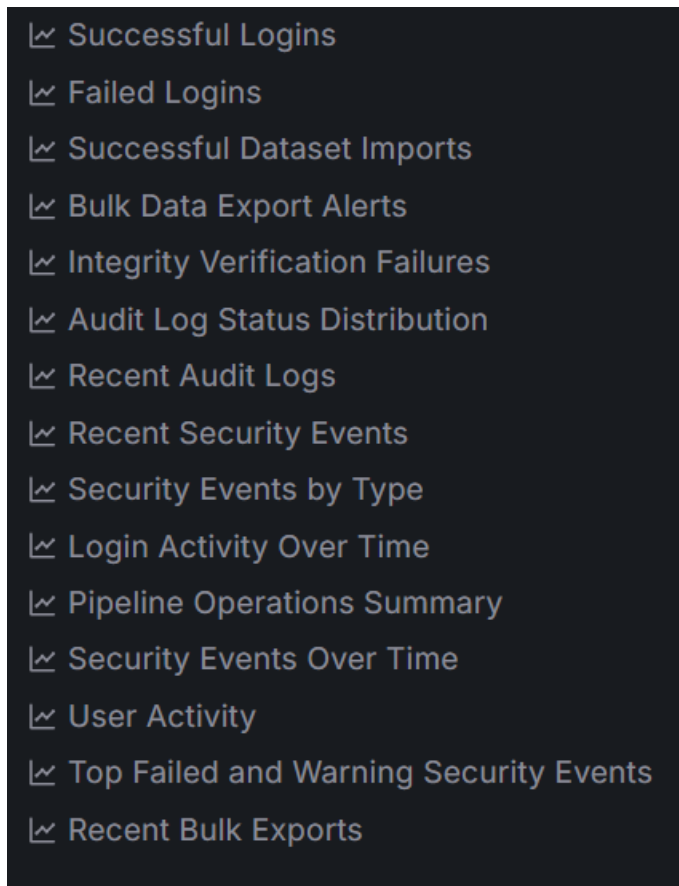


Figure 6.12: the full list of configured Grafana panels.

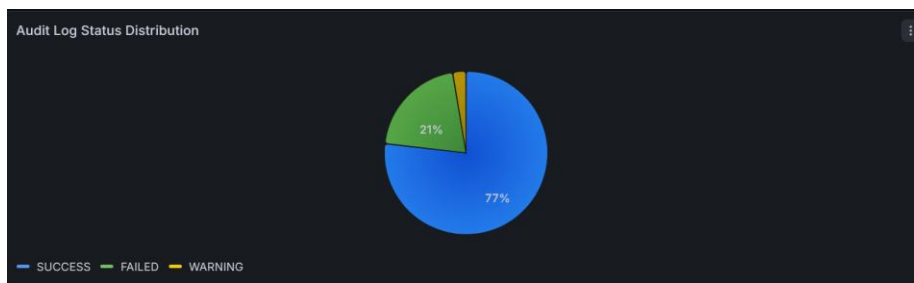


Figure 6.13: Audit Log Status Distribution (SUCCESS / FAILED / WARNING).

Time	User	Action	Status
2026-07-17 11:34:09	auditor	Viewed Security Events	SUCCESS
2026-07-17 11:34:04	auditor	User Login	SUCCESS
2026-07-17 11:25:02	admin	Viewed Security Events	SUCCESS
2026-07-17 11:24:56	admin	Bulk Data Export Detected	WARNING
2026-07-17 11:24:56	admin	Data Export	SUCCESS
2026-07-17 11:24:35	admin	User Login	SUCCESS
2026-07-17 11:23:07	admin	Viewed Security Events	SUCCESS

Figure 6.14: Recent Audit Logs panel.

Time	User	Security Event	Status
2026-07-17 11:24:56	admin	Bulk Data Export Detected	WARNING
2026-07-17 11:22:51	admin	Bulk Data Export Detected	WARNING
2026-07-17 11:14:23	admin	Bulk Data Export Detected	WARNING
2026-07-17 02:10:02	admin	Dataset Validation	FAILED
2026-07-17 01:40:06	admin	Dataset Imported	FAILED
2026-07-17 01:28:06	admin	Dataset Integrity Verification	FAILED

Figure 6.15: Recent Security Events panel.

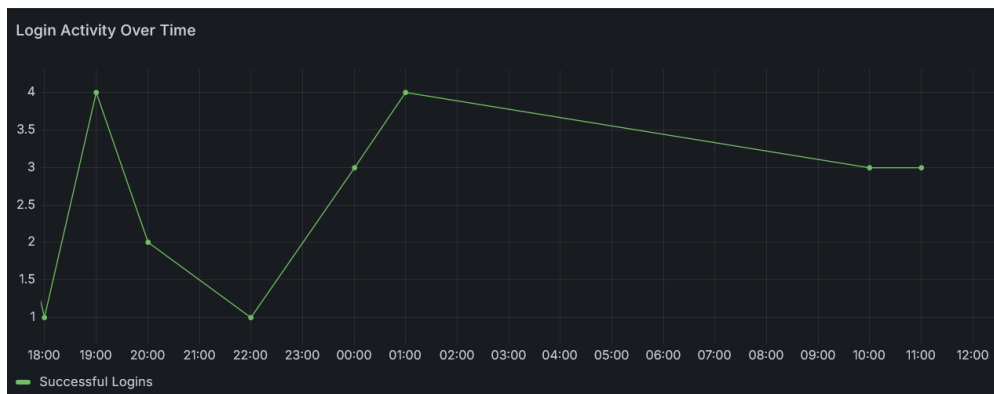


Figure 6.16: Login Activity Over Time.

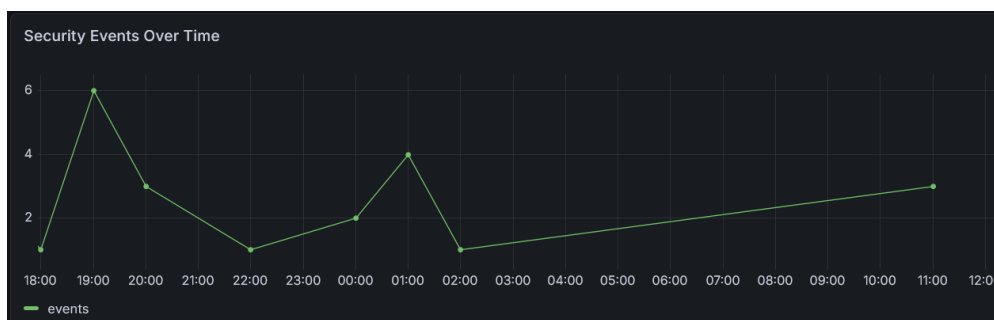


Figure 6.17: Security Events Over Time.

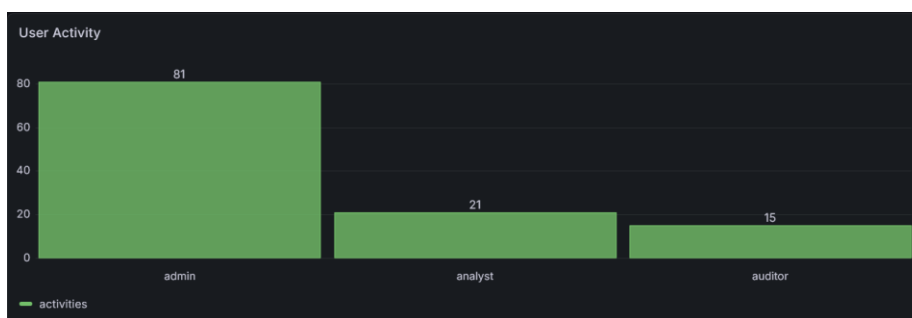


Figure 6.18: User Activity by role (admin, analyst, auditor).

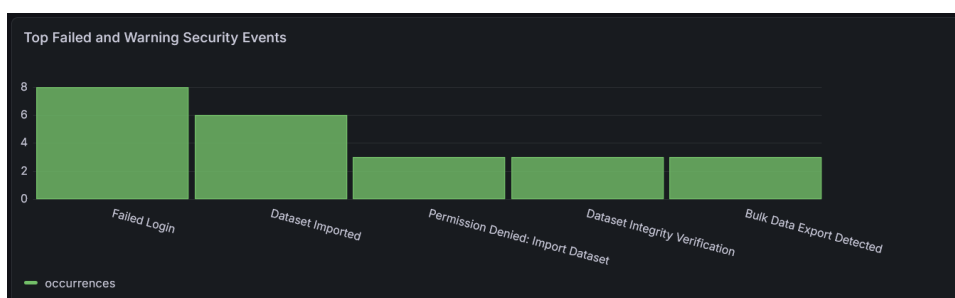


Figure 6.19: Top Failed and Warning Security Events.

Recent Bulk Exports		
log_time	username	action
2026-07-17 11:24:56	admin	Bulk Data Export Detected
2026-07-17 11:22:51	admin	Bulk Data Export Detected
2026-07-17 11:14:23	admin	Bulk Data Export Detected

Figure 6.20: Recent Bulk Exports panel.

6.8 Evidence Organization

All raw screenshots captured during development and testing were organized into labeled folders by test and incident, shown below, to support traceability between this report and the submitted evidence files.

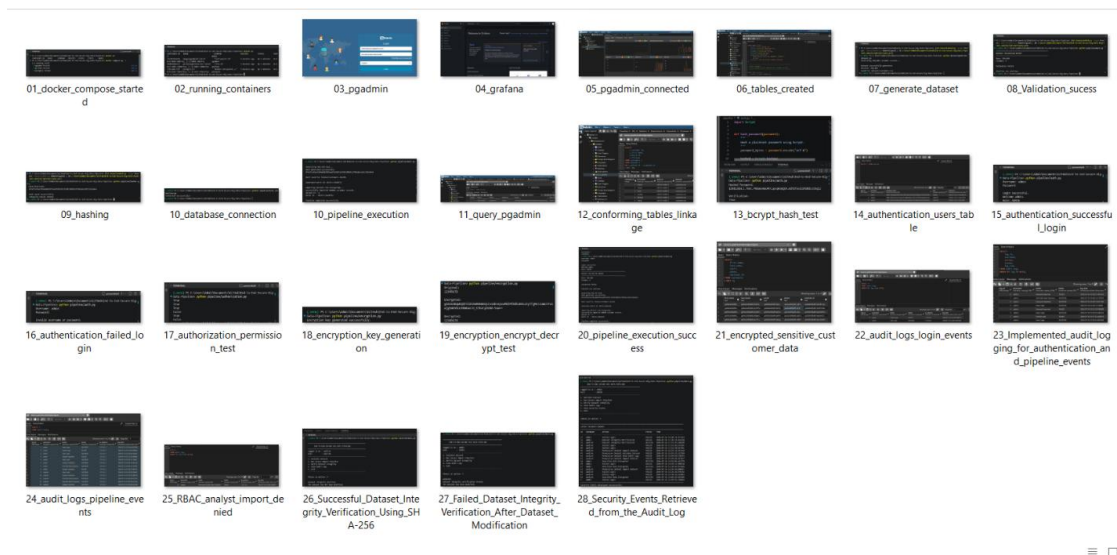


Figure 6.21: folder structure used to organize raw terminal, pgAdmin, and test evidence during development.

7. Risk Assessment Table

The following risks were identified during implementation and testing. They reflect the current state of the system as a semester project prototype, not a production deployment.

#	Risk	Impact	Likelihood	Recommended Control
1	Encryption key (keys/secret.key) is stored as a plaintext file on disk with no access restriction or rotation.	High	Medium	Move to a managed secrets store or KMS (e.g. HashiCorp Vault, AWS KMS); restrict file permissions; implement periodic key rotation with re-encryption.
2	Database credentials are hardcoded in config.py, which risks being committed to source control.	High	Medium	Load credentials from environment variables or a .env file excluded via .gitignore; use a secrets manager in production.
3	No login rate limiting or account lockout: repeated failed logins are logged but not blocked.	Medium	Medium	Add exponential backoff or temporary lockout after N consecutive failed attempts per username/IP.
4	audit_logs and security_events are stored in the same database and could theoretically be altered by anyone with direct DB (superuser) access.	High	Low	Write audit logs to an append-only or write-once store, or replicate them to a separate log-only database with restricted write access.
5	No multi-factor authentication; a single compromised password grants full role access.	Medium	Medium	Add MFA (TOTP or similar) at least for the Admin role.
6	pandas/psycopg2 raises a SQLAlchemy compatibility warning during export, indicating an unsupported connection pattern.	Low	High	Wrap the psycopg2 connection with SQLAlchemy's create_engine() before passing it to pandas.read_sql().
7	No automated backup/recovery process for the PostgreSQL data volume.	High	Medium	Schedule regular pg_dump backups (or volume snapshots) and periodically test restoration.
8	Grafana and pgAdmin are exposed on local ports with default/simple credentials and no TLS.	Medium	Medium	Enforce strong unique credentials, enable HTTPS/TLS, and restrict network access to trusted hosts.
9	Exported CSV files (exports/*.csv) contain decrypted customer data with no additional protection once exported.	High	Medium	Encrypt export files at rest, restrict the exports/directory's permissions, and log/expire exported files automatically.
10	Security event review is currently manual (dashboard/CLI review) rather than actively alerting.	Medium	High	Integrate Grafana alerting (or a SIEM) to push real-time notifications for bulk exports, integrity failures, and repeated permission denials.

8. Conclusion and Recommendations

8.1 Vulnerabilities Identified

Testing confirmed that the core security controls, including authentication, RBAC, encryption, integrity verification, audit logging, and bulk export detection, all function correctly under both normal and adverse conditions. The vulnerabilities identified (detailed in Section 7) are consistent with a semester prototype rather than a hardened production system: secrets management (encryption key and database credentials) is the most significant gap, followed by the absence of login rate limiting, automated backups, and tamper-resistant audit storage.

8.2 Remaining Risks

- A compromised admin account currently has no additional barrier (MFA) beyond a single password.
- The encryption key and database credentials, if leaked, would allow full decryption/access without further obstruction.
- Audit logs, while comprehensive, are not cryptographically tamper-evident and could be altered by a database administrator.
- Security event review depends on someone actively checking the CLI, GUI, or Grafana dashboard: there is no push alerting yet.

8.3 Recommended Improvements

- Adopt a secrets manager (Vault, AWS/Azure/GCP KMS, or at minimum environment variables) for the encryption key and database credentials.
- Add account lockout or rate limiting after repeated failed logins.
- Add MFA for privileged (Admin) accounts.
- Make audit logging append-only (e.g. via database triggers revoking UPDATE/DELETE on audit_logs, or shipping logs to a separate write-once store).
- Fix the pandas/SQLAlchemy connection warning by wrapping psycopg2 connections with SQLAlchemy's create_engine().
- Automate PostgreSQL backups and periodically test restoration.
- Enable TLS and enforce strong credentials on pgAdmin and Grafana; restrict their network exposure.
- Encrypt exported CSV files at rest and apply a retention/expiry policy to the exports/ directory.
- Replace manual dashboard review with active alerting (Grafana alert rules or a SIEM integration) for bulk exports, integrity failures, and repeated permission denials.
- Continue building out the desktop GUI (e.g. an embedded Grafana panel via pywebview) now that the core screens are complete.

8.4 Overall Assessment

The system successfully demonstrates the full data lifecycle required by the assignment: secure collection, encrypted storage, role-based access control, real-time monitoring, and integrity protection, including working detection of three simulated incidents. With the secrets-management and alerting improvements above, the design would be substantially closer to a production-ready secure data pipeline.

9. Appendices

Appendix A: Source Code

The following source files implement the pipeline described in this report and are submitted alongside it:

- config.py: centralized configuration (database connection parameters)
- database.py: reusable PostgreSQL connection function
- auth.py: bcrypt password hashing, user creation, and login
- authorization.py: RBAC permission definitions and checks
- validator.py: dataset validation rules
- hasher.py: SHA-256 hash generation, storage, and verification
- encryption.py: Fernet key generation, encryption, and decryption
- importer.py: secure import pipeline (encrypt + bulk insert)
- exporter.py: secure export with bulk-export (DLP-style) detection
- logger.py: audit log writer
- viewer.py / security_viewer.py: audit log and security event viewers (CLI)
- verifier.py: dataset integrity verification (CLI)
- workflow.py: validate/import workflow orchestration (CLI)
- main.py: CLI entry point and menu
- generate_dataset.py: synthetic dataset generator (Faker)
- schema.sql: full PostgreSQL DDL for all five tables
- app.py: desktop GUI entry point (CustomTkinter)
- gui_adapters.py: bridges the GUI to the existing backend modules
- ui/: GUI screens: login, dashboard, validate, import, verify, export, audit/security log tables, Grafana launcher, and shared theme

Appendix B: Evidence Index

A full index of raw screenshot evidence (organized by test/incident folder) is included with the submission. See Figure 6.21 for the folder structure, and Sections 4 to 6 of this report for evidence mapped directly to each test and incident.

Appendix C: Demo Accounts Used in Testing

- admin: ADMIN role, full permissions
- analyst: ANALYST role, validate and verify_hash only
- auditor: AUDITOR role, verify_hash, view_logs, and view_security_events only